

TASK MANAGER APPLICATION

Full MERN Stack

Submission	Public GitHub repository with README.md and .env.example
Stack	MongoDB • Express.js • React.js • Node.js Note: You can use Any Database

1. OVERVIEW & PROBLEM STATEMENT

Build a full-stack Task Manager application where users can register, log in, and manage their personal tasks. The application should be fully functional with a React frontend communicating with an Express REST API backed by MongoDB.

Each task must contain the following fields:

- Title — short, descriptive task name
- Description — detailed notes about the task
- Status — one of: pending, in-progress, or done
- Due Date — a date field for the task deadline
- Created At — auto-generated timestamp

Critical Constraint: Tasks must be scoped to the logged-in user. No user should be able to view, edit, or delete another user's tasks.

2. BACKEND REQUIREMENTS (Node.js + Express + MongoDB)

2.1 Data Models

- **User Model**
 - Fields: name, email, password (must be hashed using bcryptjs)
 - Email must be unique and validated before saving
- **Task Model**
 - Fields: title, description, status (enum), dueDate, createdAt
 - Must reference the User model via a userId foreign key

2.2 Authentication Routes

- POST /api/auth/register — create a new user (validate, hash password, return JWT)

- POST /api/auth/login — validate credentials, return signed JWT
- GET /api/auth/me — protected route returning the logged-in user's profile

2.3 Task CRUD Routes (all protected)

- GET /api/tasks — fetch all tasks belonging to the authenticated user
- POST /api/tasks — create a new task for the authenticated user
- PUT /api/tasks/:id — update a task (only if it belongs to the user)
- DELETE /api/tasks/:id — delete a task (only if it belongs to the user)

2.4 Auth Middleware

- Create a middleware function that extracts and verifies the JWT from the Authorization header
- Attach the decoded user object to req.user for downstream routes
- Return 401 Unauthorized if the token is missing or invalid

2.5 Validation & Error Handling

- Validate all required fields before inserting into the database
- Return meaningful HTTP status codes: 200, 201, 400, 401, 403, 404, 500
- Use try/catch on all async route handlers — never leave unhandled promise rejections

3. FRONTEND REQUIREMENTS (React.js)

3.1 Authentication Pages

- Register Page — form with name, email, password, and confirm password fields
- Login Page — form with email and password, with client-side validation
- Redirect to dashboard on successful login; show error messages on failure

3.2 Routing & Protected Routes

- Use React Router for navigation
- Create a PrivateRoute component that redirects unauthenticated users to /login
- Store the JWT in localStorage and read it on app initialization

3.3 Task Management UI

- Task List View — display all tasks as cards or a list with title, status badge, and due date
- Filter Bar — filter tasks by status: All / Pending / In-Progress / Done
- Add Task — open a modal or form to create a new task
- Edit Task — pre-populate the form with existing task data for editing
- Delete Task — show a confirmation prompt before deleting

3.4 API Communication

- Use Axios or the Fetch API to communicate with the backend
- Attach the JWT token to every API request via a request interceptor or custom hook
- Handle and display API errors clearly in the UI (not just console.log)

3.5 React Quality Expectations

- Use functional components with hooks — no class components
- Manage state with useState and useEffect correctly
- Avoid unnecessary prop drilling — use Context API or a simple state pattern for auth state
- Show loading spinners during API calls and meaningful empty states when no tasks exist

4. FOCUS AREAS TESTED

JWT Auth	REST API Design	MongoDB CRUD
React Hooks	Protected Routes	Error Handling
Async/Await	Code Structure	UX Basics

5. SUBMISSION GUIDELINES

5.1 GitHub Repository

- Push all code to a public GitHub repository
- The repo must contain two root-level folders: /client and /server
- Do not commit node_modules, .env files, or any API keys
- Include a .env.example in both /client and /server, listing all required env variables

5.2 README.md (Mandatory)

The README.md must include all of the following:

- Project overview in 2–3 sentences
- Prerequisites (Node.js version, MongoDB setup)
- Step-by-step instructions to run the backend locally
- Step-by-step instructions to run the frontend locally
- All environment variables explained (what each one is for)
- Any assumptions made or known limitations

5.3 Deadline

Deadline	24 hours from the time the assignment is shared
----------	---

5.4 Optional but Valued

- A short Loom or screen recording (2–3 minutes), walking through the running app
- Postman collection or curl examples for testing the API
- Deployment on a free tier (Render, Railway, Vercel) with a live URL